

PLANIFICACIÓN Y CRONOGRAMA DEL PROYECTO ALEX

Sistema Web de Asistencia Virtual en Salud y Emergencias

PORTADA

Universidad Tecnológica de Pereira
Facultad de Ingenierías
Programa: Tecnología en Desarrollo de Software

Asignatura: Administración y Planeación de Proyectos Software
Semestre: 2026-I (Febrero - Mayo 2026)

TÍTULO DEL PROYECTO

ALEX: Asistente Virtual de IA Especializado en Salud y Emergencias

Subtítulo: Plataforma Web para Orientación Médica y Primeros Auxilios en Colombia

Autores: [Nombres del equipo]
Fecha de Entrega: 20 de mayo de 2026
Versión: 1.0

TABLA DE CONTENIDO

1. Introducción y Justificación
2. Objetivos
3. Alcance del Proyecto
4. Metodología Elegida
5. WBS - Estructura de Desglose del Trabajo
6. Cronograma de Entregas
7. Gestión de Riesgos
8. Métricas y Calidad
9. Gestión de Costos
10. Cronograma Técnico de Desarrollo (16 semanas)

- 11. Informe de Avance (Estructura)
 - 12. Productos Entregables
 - 13. Evaluación Interna del Equipo
 - 14. Anexos
-

1. INTRODUCCIÓN Y JUSTIFICACIÓN

Problema Actual

En Colombia, el sistema de salud enfrenta desafíos críticos:

- **Sobrecarga de personal médico:** Los hospitales y clínicas operan con recursos limitados
- **Demoras en la atención:** Pacientes esperan días para consultas básicas
- **Comunicación deficiente:** Falta de información oportuna entre pacientes, médicos y entidades de salud (hospitales, clínicas, EPS, IPS)
- **Acceso limitado a información confiable:** Poblaciones en zonas rurales o de difícil acceso tienen poca orientación en emergencias
- **Baja penetración de IA en salud:** A pesar de la alta demanda, pocas soluciones tecnológicas abordan emergencias médicas

Solución Propuesta

ALEX es un asistente virtual basado en IA que:

- Proporciona **orientación en primeros auxilios y emergencias 24/7**
- Ofrece **respuestas a preguntas frecuentes de salud** con información confiable
- Mejora la **comunicación médico-paciente** a través de una plataforma centralizada
- Prepara una **arquitectura escalable** para futuras integraciones con hospitales y EPS
- Reduce **demoras en atención inicial** brindando orientación inmediata

Justificación

- **Necesidad real:** Las encuestas muestran baja penetración de IA en salud, pero **alta demanda** de soluciones que ayuden en emergencias
- **Impacto social:** Potencial de alcanzar poblaciones desatendidas en toda Colombia
- **Viabilidad técnica:** Stack moderno con Docker, API REST y IA accesible (Gemini gratuito)
- **Escalabilidad:** Arquitectura preparada para futuras versiones móviles y expansión nacional
- **Contexto académico:** Proyecto ideal para aplicar conceptos de administración y planeación de proyectos software

2. OBJETIVOS

Objetivo General

Desarrollar una **plataforma web interactiva** que integre un asistente virtual de IA para proporcionar orientación en salud, primeros auxilios y emergencias, desplegada en contenedores Docker, con una arquitectura API-first que permita escalabilidad futura hacia dispositivos móviles.

Objetivos Específicos

1. **Diseñar y implementar una API REST** escalable que sirva tanto para la plataforma web actual como para futuras aplicaciones móviles
 2. **Desarrollar una interfaz web responsive** usando React.js que permita consultas de salud mediante chat e integración de voz
 3. **Integrar APIs de IA** (Gemini/ChatGPT) para proporcionar respuestas contextuales y personalizadas a consultas médicas
 4. **Implementar autenticación y gestión de usuarios** con roles (paciente, médico, administrador) en la plataforma
 5. **Crear un catálogo de guías de primeros auxilios** precargadas en la base de datos con búsqueda y filtros por categoría
 6. **Establecer contenedores Docker** para facilitar el despliegue, reproducibilidad y facilitar escalabilidad de la solución
 7. **Registrar consultas y generar métricas básicas** de uso para evaluar adopción y calidad del servicio
 8. **Documentar completamente** tanto la arquitectura técnica como el manual de usuario para facilitar futuras expansiones
-

3. ALCANCE DEL PROYECTO

Incluye (MVP Web - Semestre)

- ✓ **Aplicación web responsive** (funciona en desktop, tablet, navegadores móviles)
- ✓ **Chat interactivo con IA** para consultas de salud (texto)
- ✓ **Entrada de voz** mediante Web Speech API (nativa del navegador)
- ✓ **Catálogo de guías de primeros auxilios** con búsqueda y filtros
- ✓ **Autenticación de usuarios** (registro, login) con JWT
- ✓ **Historial de consultas personales** para cada usuario
- ✓ **Dashboard de métricas básicas** (total consultas, categorías más consultadas)
- ✓ **Base de datos PostgreSQL** en contenedor Docker
- ✓ **Backend API REST** en contenedor Docker
- ✓ **Frontend web** en contenedor Docker
- ✓ **Documentación técnica** (instalación, deployment, API endpoints)
- ✓ **Manual de usuario** (guía de uso para pacientes y administradores)

No Incluye (Fases Futuras - Documentadas en Roadmap)

- ✗ Aplicación nativa Android/iOS
- ✗ Integración real con hospitales, EPS, IPS
- ✗ Interpretación de resultados médicos complejos
- ✗ Diagnósticos automatizados
- ✗ Recordatorios de citas médicas
- ✗ Notificaciones push
- ✗ Geolocalización de hospitales cercanos
- ✗ Videollamadas con médicos
- ✗ Publicación en Google Play Store / App Store
- ✗ Cumplimiento completo de normativa HIPAA/GDPR (solo fundamentos)

Restricciones

- **Duración:** 16 semanas (semestre académico 2026-I)
 - **Equipo:** 4-5 personas multidisciplinarias
 - **Tecnología:** Solo web (no móvil nativo en MVP)
 - **Infraestructura:** Uso de servicios cloud gratuitos (Gemini API, Railway/Render, Docker gratuito)
 - **Costo máximo:** ~100-200 USD (~400k-800k COP) por exceso de API calls
-

4. METODOLOGÍA ELEGIDA

Justificación

Scrum Ágil adaptado a contexto académico con hitos de entrega predefinidos

Razones de la Elección

1. **Iteraciones cortas (sprints de 2 semanas)** permiten entregas parciales cada corte académico
2. **Adaptabilidad:** Los cambios de requisitos se incorporan fácilmente entre sprints
3. **Visibilidad:** Sprint reviews semanales permiten detectar problemas temprano
4. **Aprendizaje:** Retrospectivas permiten reflexionar sobre procesos y mejorar
5. **Documentación continua:** Cada sprint genera documentación actualizada
6. **Integración con cronograma:** Los cortes académicos (semanas 9 y 12) alinean con fin de sprints

Estructura Scrum Propuesta

- **Sprint 1 (Semanas 1-4):** Iniciación, diseño, setup Docker
- **Sprint 2 (Semanas 5-9):** Backend - Autenticación e integración IA [CORTE 1]
- **Sprint 3 (Semanas 10-12):** Frontend - Chat, voz, guías [CORTE 2]
- **Sprint 4 (Semanas 13-16):** Integración, testing, documentación final

Reuniones Ágiles

- **Daily standup:** 15 min cada lunes, miércoles y viernes (3 días/semana)
- **Sprint planning:** Inicio de cada sprint (2 horas)
- **Sprint review:** Fin de cada sprint, demostración de funcionalidades (1.5 horas)
- **Sprint retrospectiva:** Después de cada review (1 hora)

5. WBS - ESTRUCTURA DE DESGLOSE DEL TRABAJO

ALEX - Sistema Web de Salud

|

| — 1. INICIACIÓN Y PLANIFICACIÓN

| | — 1.1 Formación del equipo y asignación de roles

| | — 1.2 Definición de objetivos y alcance

- | |— 1.3 Capacitación en tecnologías (Docker, React, Node.js)
- | |— 1.4 Configuración de repositorio Git y herramientas colaborativas
- | |— 1.5 Creación de documentación inicial del proyecto

|— 2. ANÁLISIS Y DISEÑO

- | |— 2.1 Análisis de requisitos funcionales y no funcionales
- | |— 2.2 Diseño de arquitectura de 3 capas (Frontend, Backend, BD)
- | |— 2.3 Diseño de base de datos (ER, esquema PostgreSQL)
- | |— 2.4 Diseño de interfaz de usuario (mockups, prototipo en Figma)
- | |— 2.5 Definición de endpoints API REST (Swagger)
- | |— 2.6 Análisis de riesgos y planes de mitigación

|— 3. CONFIGURACIÓN DE INFRAESTRUCTURA DOCKER

- | |— 3.1 Crear Dockerfile para backend (Node.js)
- | |— 3.2 Crear Dockerfile para frontend (React)
- | |— 3.3 Crear docker-compose.yml para orquestación
- | |— 3.4 Configurar volúmenes persistentes para PostgreSQL
- | |— 3.5 Configurar redes entre contenedores
- | |— 3.6 Testing local de contenedores

|— 4. DESARROLLO BACKEND

- | |— 4.1 Setup de proyecto Express.js con Prisma

- | |— 4.2 Implementar autenticación (JWT, registro, login)
- | |— 4.3 Crear modelos de datos en Prisma
- | |— 4.4 Implementar endpoints de usuarios
- | |— 4.5 Integración con API de Gemini (chatbot IA)
- | |— 4.6 Endpoints para consultas (crear, obtener, listar)
- | |— 4.7 Endpoints para primeros auxilios (CRUD)
- | |— 4.8 Endpoints para métricas
- | |— 4.9 Validación y manejo de errores
- | |— 4.10 Testing de API (Postman/automatizado)

|

|— 5. DESARROLLO FRONTEND

- | |— 5.1 Setup de proyecto React con Vite
- | |— 5.2 Diseño e implementación de página de inicio
- | |— 5.3 Implementar autenticación (registro, login, logout)
- | |— 5.4 Crear componente Chat con IA
- | |— 5.5 Implementar Web Speech API (entrada de voz)
- | |— 5.6 Crear catálogo de primeros auxilios
- | |— 5.7 Implementar historial de consultas
- | |— 5.8 Dashboard de métricas básicas
- | |— 5.9 Diseño responsive para múltiples dispositivos
- | |— 5.10 Optimización de rendimiento
- | |— 5.11 Testing de funcionalidades

|

| — 6. INTEGRACIÓN FRONTEND-BACKEND

| | — 6.1 Conectar frontend con API REST

| | — 6.2 Manejo de tokens JWT en cliente

| | — 6.3 Interceptores para autenticación

| | — 6.4 Sincronización de estados entre componentes

| | — 6.5 Testing end-to-end

| | — 6.6 Debugging y ajustes de performance

|

| — 7. TESTING Y CALIDAD

| | — 7.1 Pruebas unitarias (backend y frontend)

| | — 7.2 Pruebas de integración

| | — 7.3 Pruebas de seguridad (validación de inputs, SQL injection)

| | — 7.4 Pruebas de usabilidad

| | — 7.5 Pruebas de compatibilidad en navegadores

| | — 7.6 Pruebas de carga y rendimiento

| | — 7.7 Reporte de bugs y correcciones

|

| — 8. DOCUMENTACIÓN

| | — 8.1 Manual de usuario (cómo usar la plataforma)

| | — 8.2 Manual técnico (instalación, deployment)

| | — 8.3 Documentación de API (endpoints, ejemplos)

| └─ 8.4 Documentación de arquitectura (diagramas, decisiones)

| └─ 8.5 Guía de desarrollo (cómo extender el código)

| └─ 8.6 README.md con instrucciones de deployment

| └─ 8.7 Lecciones aprendidas y recomendaciones futuras

|

└─ 9. CIERRE DEL PROYECTO

└─ 9.1 Presentación final del sistema

└─ 9.2 Demo en vivo ante el curso

└─ 9.3 Entrega de código fuente y documentación

└─ 9.4 Reflexión del equipo (fortalezas, debilidades, lecciones)

└─ 9.5 Recomendaciones para futuras iteraciones (versión móvil, expansión)

6. CRONOGRAMA DE ENTREGAS (SEGÚN SYLLABUS UNIVERSITARIO)

Cronograma Académico del Semestre 2026-I

Semana	Fechas	Actividad Académica Requerida	Entregable	Responsable
3	16-18/Feb	Inicio del proyecto: formación de equipos, elección de tema y roles	Formación completada, roles asignados, repositorio Git creado	Líder de proyecto
4	23-25/Feb	Título, Introducción y Justificación	Documento con título, introducción,	Todo el equipo

Semana	Fechas	Actividad Académica Requerida	Entregable	Responsable
		(documento inicial)	justificación (2-3 pág)	
5	02-04/Mar	Objetivos y Alcance del proyecto	Objetivos generales y específicos, alcance (incluye/excluye) (2 pág)	Gerente de proyectos
6	09-11/Mar	Metodología elegida y WBS (estructura de desglose del trabajo)	Documento de metodología Scrum y WBS detallado (3 pág)	Arquitecto del proyecto
7	16-18/Mar	Cronograma preliminar (MSProject/Trello/Gantt)	Diagrama de Gantt en Trello/MS Project o similar (visual)	Gerente de proyectos
8	23-25/Mar	Gestión de Riesgos (matriz)	Matriz de riesgos con probabilidad, impacto y mitigación (1-2 pág)	Analista de riesgos
9	30/Mar-01/Abr	CORTE DE AVANCE 1: requisitos, diseño inicial, prototipo básico	Entrega 1: Documento consolidado (Semanas 4-8) + Mock-ups UI/Diagrama ER/Docker setup inicial	Todo el equipo
10	06-08/Abr	Métricas y Calidad	Definición de métricas de éxito, criterios de	QA Lead

Semana	Fechas	Actividad Académica Requerida	Entregable	Responsable
			aceptación (2 pág)	
11	13-15/Abr	Gestión de Costos (estimaciones)	Presupuesto estimado, recursos, asignación de personal (1-2 pág)	Contador del proyecto
12	20-22/Abr	Corte de Avance 2: desarrollo intermedio, pruebas iniciales	Entrega 2: Software con ~60% funcionalidades, API funcionando, primeras pruebas	Developers + QA
13	27-29/Abr	Resultados preliminares (versión casi completa del sistema)	Sistema con 90% de funcionalidades implementadas, bugs menores	Developers
14	04-06/May	Documentación del Proyecto (borrador de manual de usuario y técnico)	Manuales en borrador, API docs, arquitectura documentada	Tech Writer
15	11-13/May	Versión Final del Proyecto (software completo + documentación final)	Entrega 3: Sistema funcional 100%, documentación completa, código en GitHub	Todo el equipo
16	18-20/May	Presentación y Evaluación	Presentación (20 min), demo viva,	Todo el equipo

Semana	Fechas	Actividad Académica Requerida	Entregable	Responsable
		Final: defensa del proyecto + reflexión del equipo	reflexión del equipo, código fuente	

7. CRONOGRAMA TÉCNICO DE DESARROLLO (16 SEMANAS DETALLADO)

SPRINT 1: INICIACIÓN, DISEÑO Y SETUP DOCKER (Semanas 1-4)

Semana 1-2: Iniciación y Planificación

Lunes 16 de Feb - Miércoles 18 de Feb

Tarea	Duración	Responsable	Entregable
Formación de equipo, asignación de roles	2h	Líder	Documento de roles
Kickoff meeting: explicación de requerimientos	1h	PM	Acta de reunión
Capacitación Docker básica	4h	Tech Lead	Tutorial completado
Capacitación React.js esencial	4h	Frontend Lead	Ejemplos funcionales
Setup repositorio Git + rama develop	2h	DevOps	Repo listo con branch protection
Total semana 1	13h	-	-

Lunes 23 de Feb - Miércoles 25 de Feb

Tarea	Duración	Responsable	Entregable
Análisis de requisitos funcionales (grupo)	3h	Todo equipo	Documento de requisitos
Análisis de requisitos no funcionales	2h	Tech Lead	Requerimientos de rendimiento
Documentar problema y justificación	2h	PM	Documento intro/justificación
Setup local de Docker y docker-compose	4h	DevOps	Environment listo
Total semana 2	11h	-	-

Semana 3-4: Diseño de Arquitectura

Lunes 02 de Mar - Miércoles 04 de Mar

Tarea	Duración	Responsable	Entregable
Diseño de base de datos (diagrama ER)	4h	Backend Lead	Diagrama en DbDiagram.io
Crear esquema Prisma inicial	3h	Backend Dev	schema.prisma
Documentar endpoints API REST (Swagger)	3h	Backend Lead	Swagger YAML
Crear mockups UI en Figma	5h	Frontend Lead	5 pantallas principales
Total semana 3	15h	-	-

Lunes 09 de Mar - Miércoles 11 de Mar

Tarea	Duración	Responsable	Entregable
Definir stack tecnológico final	2h	Tech Lead	Documento de decisiones
Crear Dockerfile para backend	3h	DevOps	Dockerfile funcional
Crear Dockerfile para frontend	3h	DevOps	Dockerfile funcional
Crear docker-compose.yml	4h	DevOps	docker-compose.yml listo
Documento WBS completo	3h	PM	WBS detallado
Total semana 4	15h	-	-

Milestones Sprint 1:

-  Docker environment completamente funcional
-  Arquitectura de 3 capas documentada
-  Base de datos diseñada
-  API endpoints definidos
-  UI mockups aprobados

SPRINT 2: BACKEND E INTEGRACIÓN IA (Semanas 5-9)

Semana 5: Setup Backend y Autenticación (Parte 1)

Lunes 16 de Mar - Miércoles 18 de Mar

Tarea	Duración	Responsable	Entregable
Setup proyecto Express.js con Prisma	3h	Backend Lead	Proyecto boilerplate listo
Crear modelos Prisma (Usuario, Consulta, Contenido)	4h	Backend Dev 1	models en schema.prisma

Tarea	Duración	Responsable	Entregable
Implementar JWT y autenticación	5h	Backend Dev 1	Funciones auth.js
Crear endpoints POST /auth/register y /auth/login	4h	Backend Dev 2	Endpoints funcionando
Total semana 5	16h	-	-

Semana 6: Autenticación (Parte 2) e Integración IA

Lunes 23 de Mar - Miércoles 25 de Mar

Tarea	Duración	Responsable	Entregable
Endpoint GET /auth/me (verificar token)	2h	Backend Dev 1	Endpoint funcionando
Middleware de autenticación	3h	Backend Dev 1	middleware/authMiddleware.js
Integración API Gemini (obtener API key)	2h	Backend Lead	Credenciales configuradas
Crear servicio de IA (iaService.js)	5h	Backend Dev 2	Llamadas a Gemini funcionales
Crear endpoint POST /api/consultas	4h	Backend Dev 2	Endpoint recibe pregunta, responde IA
Total semana 6	16h	-	-

Semana 7: API de Consultas y Contenido

Lunes 30 de Mar - Miércoles 01 de Abr

Tarea	Duración	Responsable	Entregable
Endpoints GET /api/consultas (listar)	3h	Backend Dev 1	Endpoint con paginación
Endpoints GET /api/consultas/:id	2h	Backend Dev 1	Endpoint funcionando
Endpoints DELETE /api/consultas/:id	2h	Backend Dev 1	Soft delete implementado
Precargar contenido de primeros auxilios	4h	Backend Dev 2	seed.js con 25 guías
Endpoints GET /api/primeros-auxilios	3h	Backend Dev 2	Búsqueda y filtros
Testing de API en Postman	4h	QA	20+ casos de prueba
Total semana 7	18h	-	-

Semana 8: Endpoints Finales de Backend

Lunes 06 de Abr - Miércoles 08 de Abr

Tarea	Duración	Responsable	Entregable
Endpoint GET /api/metricas/resumen	3h	Backend Dev 1	Dashboard data
Validación y manejo de errores globales	4h	Backend Lead	error handler middleware
CORS y seguridad basic (helmet)	3h	Backend Dev 1	Headers seguros
Documentación de API en Swagger	4h	Tech Writer	Swagger UI interactiva
Migración de BD en Prisma	2h	DevOps	Migrations en git

Tarea	Duración	Responsable	Entregable
Testing de seguridad (SQL injection, validación)	4h	QA	Reporte de vulnerabilidades
Total semana 8	20h	-	-

Semana 9: Testing Backend Integral + CORTE 1

Lunes 13 de Abr - Miércoles 15 de Abr

Tarea	Duración	Responsable	Entregable
Pruebas unitarias de funciones críticas	6h	QA + Backend	Jest test suite
Pruebas de integración de API	6h	QA	Supertest tests
Testing de llamadas a Gemini	4h	Backend Dev 2	Casos edge testeados
Documentar backend (README, setup)	4h	Tech Writer	docs/BACKEND.md
Preparar ENTREGA 1 (doc consolidado + prototipos)	4h	PM	Reporte formal
Total semana 9	24h	-	-

CORTE DE AVANCE 1 (30 de Marzo - 1 de Abril):

Entregable:

- Documento consolidado (Introducción, Objetivos, Alcance, Metodología, WBS, Cronograma, Riesgos)
- Mockups de UI finales
- Diagrama ER de base de datos
- Docker images funcionando
- Backend API 100% funcional
- Documentación de Swagger

Estado esperado: Backend lista para ser consumida, base de datos funcionando, IA integrada

SPRINT 3: FRONTEND WEB (Semanas 10-12)

Semana 10: Setup Frontend y Autenticación

Lunes 20 de Abr - Miércoles 22 de Abr

Tarea	Duración	Responsable	Entregable
Setup React.js + Vite + Material-UI	4h	Frontend Lead	Proyecto boilerplate
Crear estructura de carpetas (pages, components, services)	2h	Frontend Lead	Estructura lista
Implementar servicio API (axios + interceptores)	4h	Frontend Dev 1	apiService.js con autenticación
Página Home / Landing	5h	Frontend Dev 2	Hero section + CTA
Página Login y Registro	6h	Frontend Dev 1	Formularios con validación
Total semana 10	21h	-	-

Semana 11: Chat y Voz

Lunes 27 de Abr - Miércoles 29 de Abr

Tarea	Duración	Responsable	Entregable
Componente Chat (historial, input, respuesta)	7h	Frontend Dev 1	ChatBot.jsx funcional

Tarea	Duración	Responsable	Entregable
Web Speech API integration (micrófono)	5h	Frontend Dev 2	VoiceInput.jsx funcionando
Historial de consultas (página)	4h	Frontend Dev 1	Página con lista de consultas
Styling responsive con TailwindCSS	4h	Frontend Lead	Mobile first UI
Testing de componentes React	4h	QA	Jest + React Testing Library
Total semana 11	24h	-	-

Semana 12: Guías y Dashboard + CORTE 2

Lunes 04 de May - Miércoles 06 de May

Tarea	Duración	Responsable	Entregable
Página Primeros Auxilios (catálogo)	6h	Frontend Dev 2	Búsqueda + filtros
Página Detalle de Guía (vista completa)	4h	Frontend Dev 2	Guía con pasos e imágenes
Dashboard de Métricas (gráficos Chart.js)	6h	Frontend Dev 1	Dashboard visuales
Componente Navbar y navegación	3h	Frontend Lead	Menú responsive
Pruebas end-to-end (Cypress)	5h	QA	E2E tests de flujos principales
Total semana 12	24h	-	-

CORTE DE AVANCE 2 (20-22 de Abril):

Entregable:

- Software con ~60-70% de funcionalidades
- Frontend conectado con Backend
- Chat funcionando con IA
- Web Speech API integrada
- Catálogo de guías operativo
- Dashboard de métricas básicas
- Pruebas iniciales completadas

Estado esperado: Sistema casi completo, falta integración final y pulido

SPRINT 4: INTEGRACIÓN, TESTING Y DOCUMENTACIÓN (Semanas 13-16)

Semana 13: Integración y Testing End-to-End

Lunes 11 de May - Miércoles 13 de May

Tarea	Duración	Responsable	Entregable
Testing completo de flujos usuario (E2E)	8h	QA	Cypress tests completos
Debugging y ajustes de performance	6h	Developers	Optimizaciones aplicadas
Pruebas de compatibilidad en navegadores	4h	QA	Chrome, Firefox, Safari
Pruebas de seguridad (OWASP)	5h	Security	Reporte de vulnerabilidades
Deployment en contenedor local	4h	DevOps	docker-compose up -d OK
Total semana 13	27h	-	-

Semana 14: Documentación

Lunes 18 de May - Miércoles 20 de May

Tarea	Duración	Responsable	Entregable
Manual de Usuario (paso a paso con capturas)	6h	Tech Writer	PDF 10-15 páginas
Manual Técnico (instalación, deployment)	6h	Tech Writer	PDF 8-10 páginas
Documentación de API (Swagger + ejemplos)	4h	Backend Lead	Swagger + postman collection
Arquitectura documentada (diagrama C4)	5h	Tech Lead	Diagrama + explicación
README.md completo del proyecto	3h	Tech Writer	README en GitHub
Guía de desarrollo futuro (cómo extender)	4h	Tech Lead	Documento de extensibilidad
Total semana 14	28h	-	-

Semana 15: Ajustes Finales

Lunes 25 de May - Miércoles 27 de May

Tarea	Duración	Responsable	Entregable
Bug fixes críticos	6h	Developers	Lista de bugs cerrados
Optimización de queries BD	4h	Backend	Índices agregados

Tarea	Duración	Responsable	Entregable
Optimización de bundle frontend (minificación)	4h	Frontend	Reducción de tamaño JS
Prueba final de despliegue local	4h	DevOps	Checklist completado
Lecciones aprendidas preliminares	3h	PM	Documento en borrador
Preparación de presentación	4h	PM + Presentador	Slides + demoscrypt
Total semana 15	25h	-	-

Semana 16: PRESENTACIÓN FINAL

Lunes 01 de Jun - Miércoles 03 de Jun

Tarea	Duración	Responsable	Entregable
Ensayo de presentación	3h	Equipo	Feedback aplicado
Demo en vivo del sistema	15 min	Presentador	Demostración funcionando
Presentación del proyecto (20 min)	20 min	Presentador	Slides + narración
Reflexión del equipo (10 min)	10 min	Todo equipo	Fortalezas, debilidades, lecciones
Preguntas de evaluadores	15 min	Equipo	Respuestas preparadas
Total presentación	63 min	-	-

ENTREGA FINAL:

-  Código fuente en GitHub (rama main)
-  Docker images y docker-compose.yml
-  Documentación completa (manuales, API, arquitectura)
-  Manual de usuario
-  Base de datos con datos de prueba
-  Informe final con lecciones aprendidas
-  Presentación ejecutiva

8. GESTIÓN DE RIESGOS

Matriz de Riesgos

#	Riesgo	Probabilidad	Impacto	Nivel	Plan de Mitigación	Responsable
1	Retraso en integración API IA (Gemini)	Media	Alto	ALTO	Validar API key temprano (Sem 6), tener endpoint mock funcionando en paralelo	Backend Lead
2	Problemas de compatibilidad de navegadores	Media	Medio	MEDIO	Testing en Chrome, Firefox, Safari desde Sem 11; usar polyfills	QA
3	Inexperiencia del equipo con Docker	Media	Medio	MEDIO	Capacitación en Sem 1-2, usar docker-compose para simplificar, stack	DevOps

#	Riesgo	Probabilidad	Impacto	Nivel	Plan de Mitigación	Responsable
					overflow como referencia	
4	Sobrecarga de datos en BD (performance)	Baja	Alto	MEDIO	Agregar índices desde diseño (Sem 3), testing de carga en Sem 13	Backend Lead
5	Cambios en requisitos a mitad de proyecto	Alta	Medio	ALTO	Metodología Scrum permite cambios entre sprints, pero requiere aprobación del PM	PM
6	Miembro del equipo se enferma/abandona	Baja	Alto	MEDIO	Documentar todo en Git, conocimiento distribuido, backup roles definidos	Líder
7	Límite de API Gemini excedido (no gratuita)	Media	Bajo	BAJO	Monitorear consumo semanalmente, usar caché de respuestas, ChatGPT	Backend Lead

#	Riesgo	Probabilidad	Impacto	Nivel	Plan de Mitigación	Responsable
					como alternativa	
8	Fallo de contenedor Docker en producción	Baja	Alto	MEDIO	Health checks configurados, volúmenes persistentes backup, documentar rollback	DevOps
9	Security: SQL injection o XSS	Baja	Crítico	ALTO	Validación en backend, prepared statements en Prisma, sanitización en frontend	Backend + Frontend
10	Falta de comunicación entre equipo	Alta	Medio	ALTO	Dailies 3x/semana, Slack para comunicación, wiki compartida	PM

Estrategias por Nivel

RIESGOS ALTOS (1, 5, 9, 10):

- Seguimiento semanal
- Escalación inmediata si se concreta
- Plan de contingencia documentado

RIESGOS MEDIOS (2, 3, 4, 6, 8):

- Revisión quincenal

- Preventivas desde el inicio
- Escalación si impacta cronograma

RIESGOS BAJOS (7):

- Monitoreo pasivo
- Revisión en retrospectiva

9. MÉTRICAS Y CALIDAD

Métricas de Éxito del Proyecto

Métrica	Meta	Cómo se Mide	Responsable
% Cumplimiento del cronograma	≥ 95%	Comparar fechas reales vs planificadas	PM
% Funcionalidades implementadas	100% (MVP)	Testing functional checklist	QA
% Casos de prueba aprobados	≥ 95%	Casos exitosos / casos totales	QA
Tiempo respuesta API (p50)	< 500ms	APM tools o logs	Backend Lead
Disponibilidad en testing	≥ 99%	Uptime en contenedores	DevOps
Cobertura de código	≥ 70%	Jest coverage report	QA
Issues de seguridad crítica	0	OWASP testing	Security
Documentación completitud	100%	Checklist de entregables	Tech Writer
Satisfacción del equipo	≥ 4/5	Post-sprint survey	PM

Criterios de Aceptación (DoD - Definition of Done)

Una funcionalidad se considera "lista" cuando:

- ☒ Código escrito según estándares del equipo
- ☒ Código revisado por 1+ compañero (code review)
- ☒ Pruebas unitarias con $\geq 70\%$ cobertura
- ☒ Integración exitosa en rama develop
- ☒ Probada en ambiente local dockerizado
- ☒ Documentada (comentarios en código + wiki)
- ☒ Sin warnings en linter (ESLint, Prettier)
- ☒ Performance aceptable (benchmarks cumplidos)

Plan de Testing

Semana 7-9: Testing backend (Postman, Jest)

Semana 11-12: Testing frontend (React Testing Library, Cypress)

Semana 13: Testing end-to-end integral (E2E Cypress)

Semana 14: Testing de seguridad y performance

Semana 15: Pruebas finales y ajustes

Tipos de pruebas:

- Unitarias: Funciones individuales
- Integración: Módulos interactuando
- E2E: Flujos completos de usuario
- Seguridad: OWASP Top 10
- Performance: Carga, estrés
- Compatibilidad: Navegadores diferentes

10. GESTIÓN DE COSTOS

Presupuesto Estimado para MVP (Semestre)

Categoría	Detalle	Costo Estimado	Notas
Infraestructura	-	-	-
	Docker Desktop	\$0	Gratuito
	PostgreSQL 15	\$0	Gratuito (contenedor)
	Node.js + npm	\$0	Gratuito

Categoría	Detalle	Costo Estimado	Notas
	React.js	\$0	Gratuito
APIs Externas	-	-	-
	Gemini API (gratuita)	\$0	50k requests/día gratis
	ChatGPT API (backup)	\$5-20	Opcional si excedes Gemini
Servicios Cloud (Opcional)	-	-	-
	Railway/Render (deploy)	\$0-25	Opcional para MVP académico
	Dominio (.com)	\$10	Opcional
Herramientas Desarrollo	-	-	-
	GitHub	\$0	Gratuito (public repo)
	Figma	\$0	Plan gratuito suficiente
	VS Code	\$0	Gratuito
	Postman	\$0	Versión gratuita
	DBeaver	\$0	Gratuito Community
Materiales Físicos	-	-	-
	Papelería/impresiones	\$20-30	Documentación física
	Meeting room (universidad)	\$0	Incluido en institución
Mano de Obra (Referencial)	-	-	-
	4 personas × 240h × \$0/h	\$0	Trabajo académico no remunerado

Categoría	Detalle	Costo Estimado	Notas
TOTAL ESTIMADO		\$35-80 USD	(~140k-320k COP)

Justificación de Costos Bajos

- **Tecnologías open-source:** React, Node.js, PostgreSQL, Docker = \$0
- **APIs freemium:** Gemini permite 50k requests/día gratis
- **Equipo académico:** No se contempla salarios (trabajo de semestre)
- **Infraestructura local:** Desarrollo en máquinas personales del equipo
- **Cloud opcional:** Para MVP es suficiente desarrollo local dockerizado

Recursos por Rol (Estimado)

Rol	Horas Totales	% Semestre	Dedicación
Project Manager	120h	30%	Coordinación, documentación
Desarrollador Backend	160h	40%	Código API REST
Desarrollador Frontend	160h	40%	React.js, UI/UX
DevOps Engineer	80h	20%	Docker, deployment
QA / Tester	100h	25%	Testing integral
Tech Writer	60h	15%	Documentación
TOTAL	~680h	~170h/persona	Aprox. 10-11h/semana por persona

11. INFORME DE AVANCE (Estructura de Entregas)

Corte de Avance 1 (Semana 9 - 30 Marzo - 1 Abril)

Documento consolidado a entregar:

1. Portada + tabla de contenido
2. Introducción y justificación (actualizada)

3. Objetivos (general + específicos)
4. Alcance (incluye/excluye/restricciones)
5. Metodología Scrum elegida
6. WBS detallado (9 componentes principales)
7. Cronograma de 16 semanas
8. Análisis de riesgos (matriz 10×4)
9. Métricas de éxito
10. Gestión de costos preliminar

Entregables técnicos:

- ☒ Mockups UI/UX (5 pantallas principales) en Figma
- ☒ Diagrama ER de base de datos
- ☒ Diagrama de arquitectura de 3 capas
- ☒ Docker images compiladas y funcionando
- ☒ Backend API 100% funcional (endpoints CRUD completos)
- ☒ Integración con Gemini validada
- ☒ Documentación Swagger de API
- ☒ Repositorio GitHub con estructura lista

Estado esperado: Backend completamente funcional, database funcionando, infraestructura Docker lista para frontend

Corte de Avance 2 (Semana 12 - 20-22 Abril)

Software operativo:

- ☒ Frontend web ~70% implementado
- ☒ Chat con IA operativo (texto + voz)
- ☒ Catálogo de primeros auxilios completo
- ☒ Autenticación usuario funcionando (registro/login)
- ☒ Historial de consultas accesible
- ☒ Dashboard de métricas básicas visualizando datos
- ☒ Responsive design comprobado en mobile/tablet/desktop

Pruebas completadas:

- ☒ 60+ casos de prueba funcionales aprobados
- ☒ Pruebas de integración backend-frontend exitosas
- ☒ Pruebas de usabilidad básicas
- ☒ Reporte de bugs menores vs críticos

Documentación:

- ☒ Manual de usuario borrador
- ☒ Manual técnico borrador
- ☒ Swagger API completo

Estado esperado: Sistema ~85% funcional, listo para ajustes finales

Entrega Final (Semana 16 - 18-20 Mayo)

Código fuente:

- ☒ Repositorio GitHub con rama main actualizada
- ☒ Todo el código documentado con comentarios
- ☒ Commits con mensajes descriptivos

Docker:

- ☒ Dockerfiles optimizados
- ☒ docker-compose.yml funcional
- ☒ Instrucciones de deployment en README

Sistema completo:

- ☒ 100% de funcionalidades MVP implementadas
- ☒ 0 bugs críticos, máximo 2-3 bugs menores documentados
- ☒ Performance optimizado (tiempo respuesta <500ms)
- ☒ Seguridad validada (OWASP)

Documentación final:

- ☒ Manual de Usuario (PDF 10-15 páginas)
- ☒ Manual Técnico (PDF 8-10 páginas)
- ☒ Documentación de API (Swagger exportado + Postman collection)
- ☒ Arquitectura documentada (diagramas C4)
- ☒ Guía de desarrollo futuro
- ☒ README.md completo

Presentación:

- ☒ Slides profesionales (15-20 slides)
- ☒ Demo en vivo del sistema (15 min)
- ☒ Reflexión del equipo y lecciones aprendidas

12. PRODUCTOS ENTREGABLES

Entregables de Software

1. Código fuente web

- Repositorio GitHub público/privado
- Frontend React.js
- Backend Node.js/Express
- Dockerfile y docker-compose.yml
- Base de datos PostgreSQL con seed data

2. Documentación técnica

- README.md con instrucciones de instalación
- Swagger/OpenAPI documentation de API
- Diagrama de arquitectura (C4 model)
- Diagrama ER de base de datos
- Guía de desarrollo para extensiones futuras

3. Documentación de usuario

- Manual de Usuario (paso a paso con capturas)
- FAQ (Preguntas frecuentes)
- Guía de primeros auxilios integrada

4. Entrega e instalación

- Instrucciones de deployment local (docker-compose up -d)
- Instrucciones de deployment en Railway/Render (opcional)
- Script de seed de datos de prueba

Entregables de Gestión de Proyectos

1. Documento de planificación (este archivo)

- Plan del proyecto completo
- Cronograma de 16 semanas
- Matriz de riesgos
- Métricas de éxito

2. Informes de avance

- Corte 1 (Semana 9): Requisitos + Diseño + Backend
- Corte 2 (Semana 12): Desarrollo intermedio
- Informe final (Semana 16): Cierre del proyecto

3. **Presentación ejecutiva**

- Slides de presentación
- Demo en vivo
- Grabación de demo (si aplica)

4. **Reflexión del equipo**

- Fortalezas del proyecto
- Debilidades encontradas
- Lecciones aprendidas
- Recomendaciones para futuras iteraciones

13. EVALUACIÓN INTERNA (REFLEXIÓN DEL EQUIPO)

A completar al finalizar el proyecto en Semana 16

(Esta sección será completada con las reflexiones reales del equipo)

Temas a reflexionar:

1. **Comunicación dentro del equipo**

- ¿Cómo fue la comunicación?
- ¿Qué canales funcionaron mejor?
- ¿Qué mejorarían para próximos proyectos?

2. **Gestión del tiempo**

- ¿Se cumplió con los cronogramas?
- ¿Qué actividades tardaron más de lo esperado?
- ¿Cómo distribuir mejor el tiempo en futuros proyectos?

3. **Trabajo en equipo**

- ¿Roles y responsabilidades fueron claros?
- ¿Equipo equilibrado en capacidades técnicas?

- ¿Hubo conflictos? ¿Cómo se resolvieron?

4. Desafíos técnicos

- ¿Cuál fue el mayor desafío técnico?
- ¿Cómo lo superaron?
- ¿Qué hubiera facilitado el desarrollo?

5. Aprendizajes clave

- ¿Qué habilidades nuevas adquirieron?
- ¿Qué conceptos de gestión de proyectos practicaron?
- ¿Qué aplicarían diferente en el futuro?

6. Satisfacción general

- ¿Calificación del proyecto 1-10?
- ¿Orgullo del resultado final?
- ¿Recomendación a otros estudiantes?

14. ANEXOS

A. Diccionario de Términos

Término	Definición
MVP	Minimum Viable Product - Versión mínima funcional
API REST	Interfaz de programación basada en HTTP
JWT	JSON Web Token - Standard de autenticación
Prisma	ORM moderno para bases de datos
Docker	Plataforma de contenedores para virtualización
React	Framework JavaScript para interfaces de usuario

Término	Definición
Express.js	Framework backend minimalista para Node.js
PostgreSQL	Base de datos relacional open-source
Scrum	Metodología ágil de gestión de proyectos
Sprint	Iteración de desarrollo de 1-4 semanas
Backlog	Lista de funcionalidades por implementar
DoD	Definition of Done - Criterios de aceptación
E2E	End-to-End - Pruebas de flujos completos
CORS	Cross-Origin Resource Sharing
OWASP	Open Web Application Security Project

B. Referencias y Recursos

Documentación oficial:

- [Docker Docs](#)
- [React Documentation](#)
- [Express.js Guide](#)
- [Prisma Documentation](#)
- [PostgreSQL Manual](#)

Tutoriales recomendados:

- Docker & Docker Compose (YouTube)
- React.js Full Course (freeCodeCamp)
- Node.js & Express for Beginners (Traversy Media)
- Web Speech API MDN Web Docs

Metodologías:

- Scrum Guide oficial
- Agile Manifesto

C. Plantilla de Retrospectiva Sprint

Semana: ____ | **Sprint:** ____

¿QUÉ SALIÓ BIEN?

¿QUÉ SALIÓ MAL?

¿QUÉ MEJORAR PARA PRÓXIMO SPRINT?

Velocidad del sprint: _____ puntos de historia completados

Tareas incompletas: _____ (razones: _____)

Cambios de alcance: Sí / No (si sí, detallar: _____)

RESUMEN EJECUTIVO

Estado del Proyecto

Proyecto ALEX - Sistema web de asistencia virtual en salud es una aplicación moderna que integra IA para orientación médica y primeros auxilios en Colombia.

Viabilidad

- ✓ **Técnica:** Stack moderno con tecnologías maduras (React, Node.js, PostgreSQL, Docker)
- ✓ **Temporal:** 16 semanas de semestre es tiempo suficiente para MVP web
- ✓ **Económica:** Presupuesto bajo (~\$50-80 USD) usando tecnologías open-source
- ✓ **De recursos:** Equipo de 4-5 personas multidisciplinarias es adecuado

Riesgos Principales

- ⚠ **Integración API IA:** Mitigado con validación temprana (Sem 6)
- ⚠ **Cambios de requisitos:** Manejado con metodología Scrum flexible
- ⚠ **Seguridad:** Pruebas OWASP desde Semana 13

Próximos Pasos

1. Semana 3-4: Setup del proyecto y capacitación del equipo
2. Semana 5-9: Desarrollo backend + CORTE 1
3. Semana 10-12: Desarrollo frontend + CORTE 2
4. Semana 13-16: Integration, testing, documentación + ENTREGA FINAL

Contacto y Escalaciones

Project Manager: [Nombre] - [Email]

Technical Lead: [Nombre] - [Email]

Escalación crítica: [Profesor/Asesor] - [Email]

Documento preparado para: Administración y Planeación de Proyectos Software

Universidad Tecnológica de Pereira - 2026-I

Versión: 1.0 | Fecha: 18 de Febrero de 2026